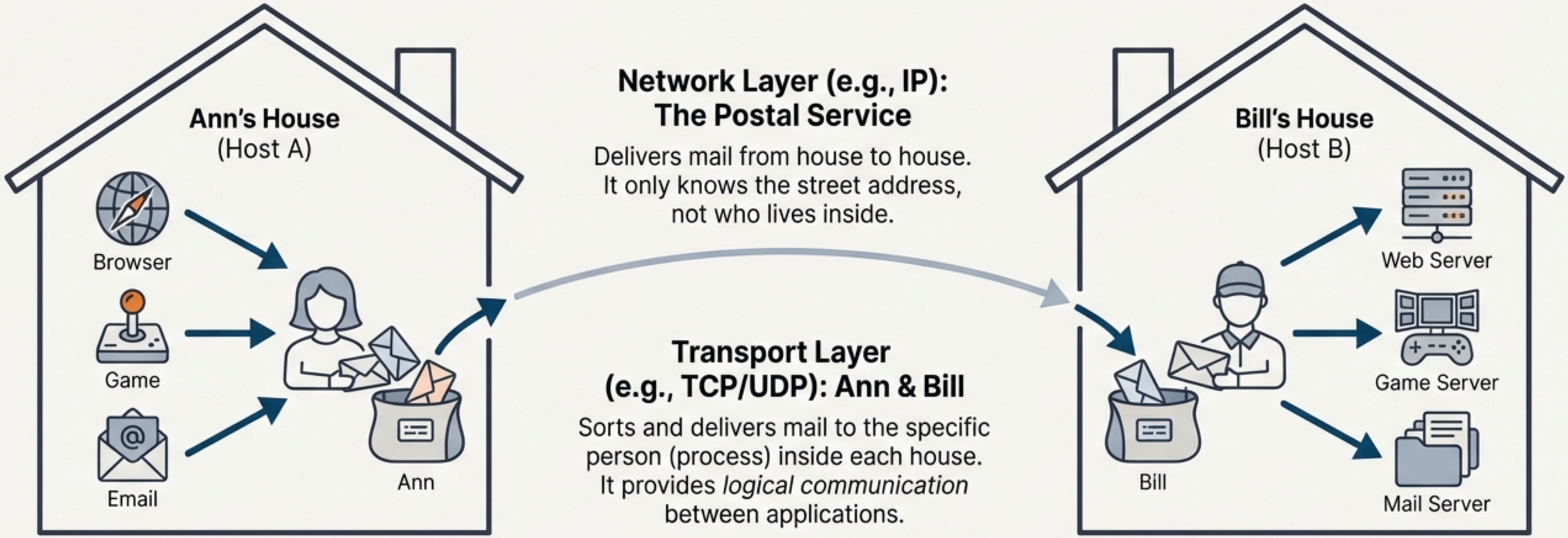
The Mission: Connecting Applications, Not Just Computers



Sender:

Breaks application messages into smaller pieces called *segments*.

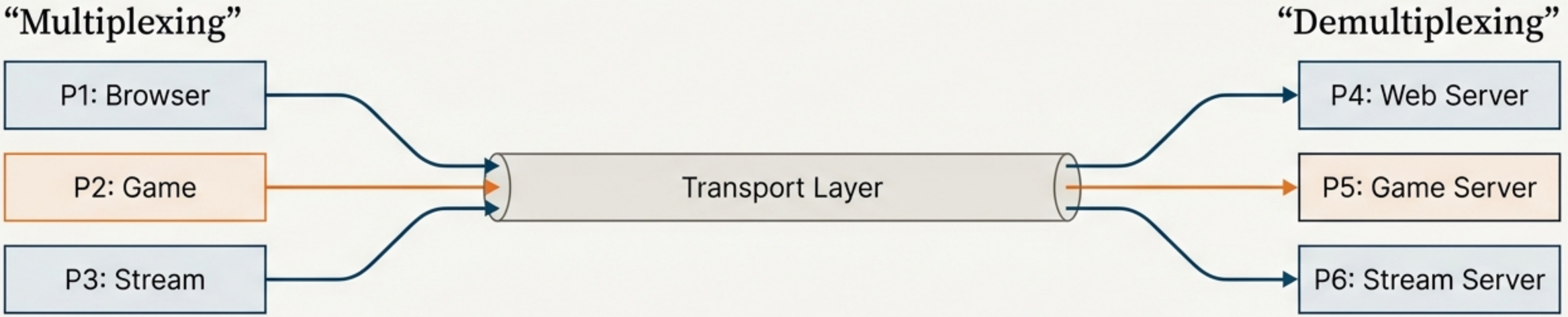
Receiver:

Reassembles segments back into messages and delivers them to the correct application.

NotebookLM

Challenge #1: Delivering to the Right Mailbox

Your computer is running a web browser, a streaming app, and a video game at the same time.
When data arrives, how does it know whether to go to Firefox, Netflix, or Skype?



How it Works: Port Numbers

Each application process is assigned a unique port number on the host. The receiving host uses the *destination port number* in the segment header to direct the data to the correct application’s socket.

Source Port # 49152	**Destination Port #** 80	Sequence Number	Acknowledgment Number	Header Length	Flags	Window Size	Checksum	Urgent Pointer	Options	Data

NotebookLM

A Tale of Two Protocols: Meet the Contenders

UDP: The Sprinter

(User Datagram Protocol)

Philosophy: Speed and simplicity above all else. A “no-frills” extension of the network’s “best-effort” service.

- **Unreliable & Unordered:** Segments can be lost or arrive out of order.
- **Connectionless:** No setup required. Just send. No initial delay.
- **Minimalist:** Small header, no connection state to manage.

TCP: The Marathon Runner

(Transmission Control Protocol)

Reliosophy: Reliability is paramount. Guarantees the job gets done correctly, no matter the obstacles.

- **Reliable & In-Order:** Guarantees delivery and correct sequencing.
- **Connection-Oriented:** A “handshake” is required to set up the connection.
- **Feature-Rich:** Includes flow control and congestion control.

NotebookLM

UDP's Philosophy: "Just Send It"

Why would anyone use an unreliable protocol?

Because for some applications, speed is more important than perfect reliability.

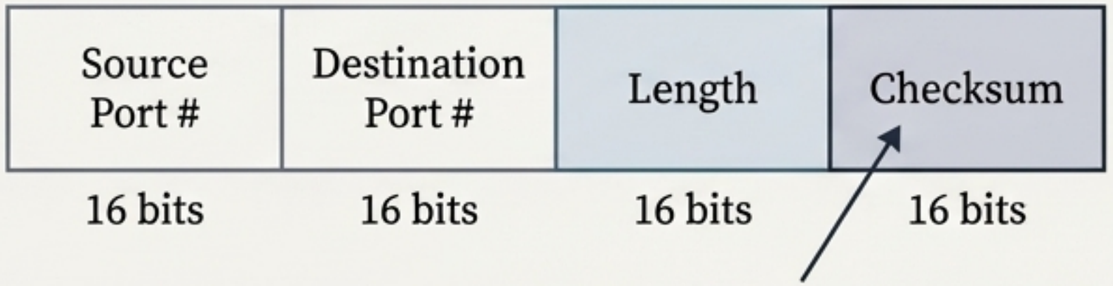
When to Use UDP

Streaming Multimedia: Tolerant to some loss (a few dropped frames in a video is better than pausing to wait). Sensitive to delay.

DNS & SNMP: Simple, fast, query-response transactions.

HTTP/3: Builds its own reliability on top of UDP's speed.

The UDP Segment Header



UDP is a bare-bones protocol. If you need more features like reliability, you must build them yourself at the application layer.

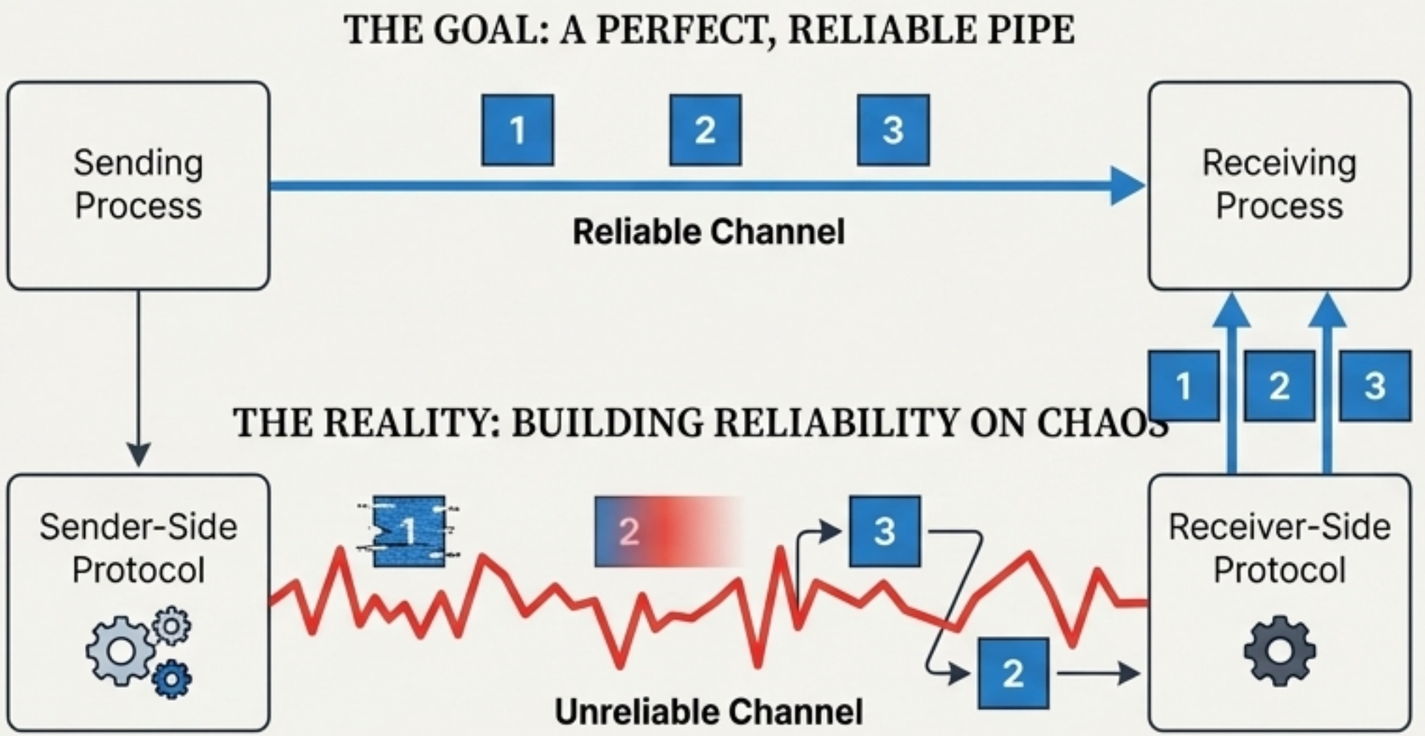
NotebookLM

Challenge #2: The Unreliable Postman

The underlying network layer is unreliable. It might:

- **Corrupt data** (flip bits in a packet).
- **Lose packets** entirely.
- **Deliver packets out of order**.

How can we build a *reliable data transfer (rdt)* service on top of an inherently unreliable channel?



Think of it as having a careful conversation over a bad phone line. You need strategies to ensure the message gets through correctly.

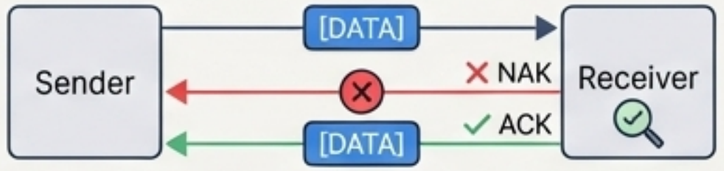
NotebookLM

Forging Reliability: A Step-by-Step Guide

Problem: Bit Errors

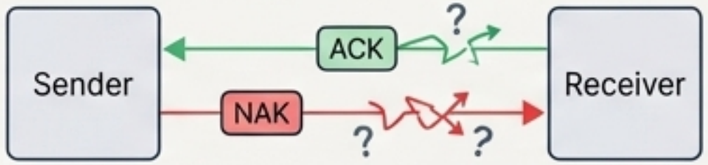
Data packets get corrupted during transmission.

Solution: Checksums and Acknowledgements (ACKs/NAKs)



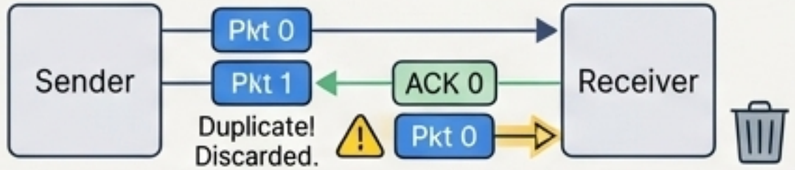
Receiver tells the sender if the packet was OK (ACK) or corrupted (NAK).

Problem: Garbled Feedback



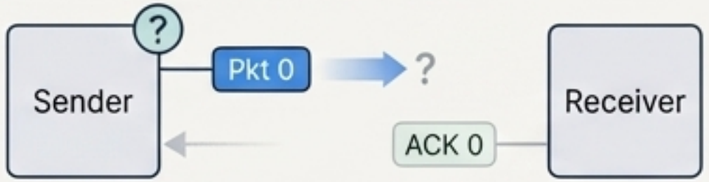
The feedback signal itself can be corrupted on the way back.

Solution: Sequence Numbers



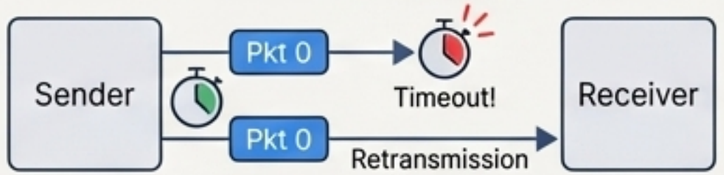
Sender adds a number to each packet (0, 1, 0, 1...).
Receiver can now detect and discard duplicate packets.

Problem: Lost Packets



Packets or acknowledgements can be lost entirely in transit.

Solution: A Countdown Timer



If an ACK isn't received before the timer expires, the sender assumes the packet was lost and retransmits.

NotebookLM

The Problem with “Stop-and-Wait”: A Performance Bottleneck

The Inefficiency of Stop-and-Wait

The sender sends one packet and does nothing until an ACK is received. On a fast network, this is incredibly wasteful.

$$U_{sender} = \frac{L/R}{RTT + L/R}$$

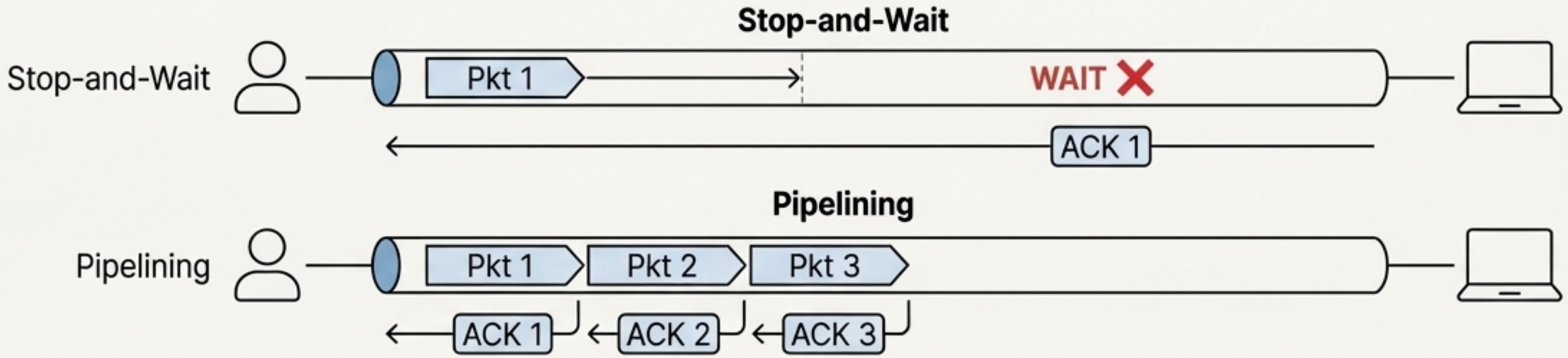
Link Speed (R): 1 Gbps
Latency: 15 ms
Round Trip Time (RTT): 30 ms
Packet Size (L): 8000 bits
Transmission Time (L/R): 8 microseconds (0.008 ms)

$$\approx 0.027\%$$

The sender is busy transmitting data for only **0.027%** of the time.

The Solution: Pipelining

Allow the sender to have multiple, “in-flight,” yet-to-be-acknowledged packets at once, filling the network “pipe”.



NotebookLM

TCP’s Toolkit for Reliable Pipelining

TCP's Core Mechanisms

TCP uses a sophisticated set of tools to manage reliable pipelining:

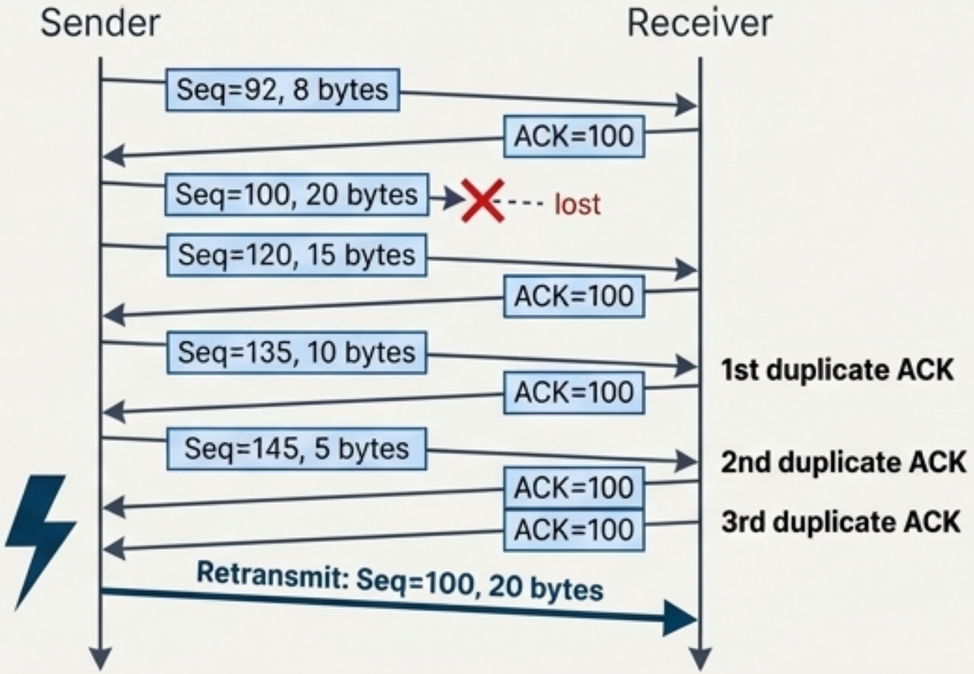
1. **Byte-Stream Sequence Numbers:** TCP numbers every single *byte* of data, not just packets. The sequence number is the first byte in the segment.
2. **Cumulative ACKs:** An ACK number from the receiver means “I have received all bytes up to this number.” This is very efficient.
3. **A Single Retransmission Timer:** TCP maintains one conceptual timer for the oldest un-ACKed segment. The timeout value is dynamically calculated based on the measured Round Trip Time (RTT).

A Clever Optimization

Fast Retransmit

Problem: Waiting for a timeout to detect a lost packet can be slow.

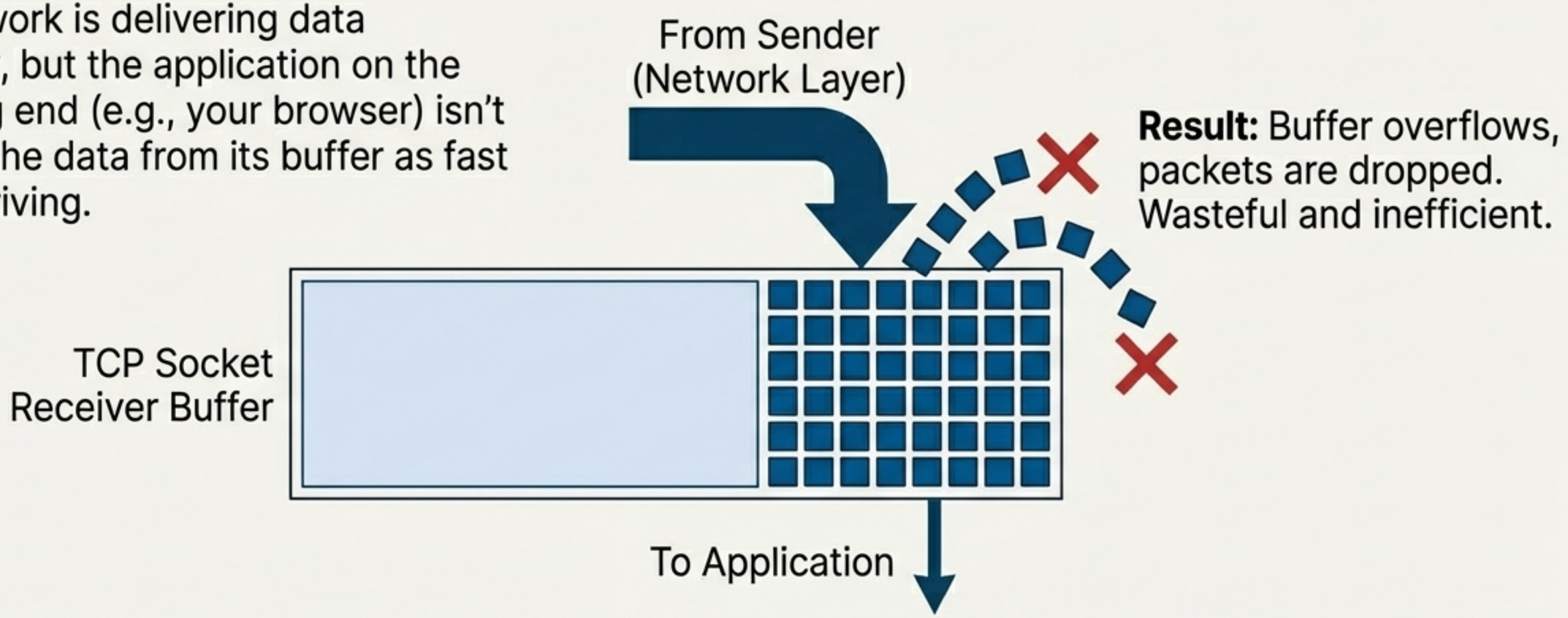
TCP's Insight: If a sender receives multiple identical ACKs, it's a strong hint that the segment *after* the acknowledged one was lost.



NotebookLM

Challenge #3: The Overwhelmed Recipient

The network is delivering data perfectly, but the application on the receiving end (e.g., your browser) isn't reading the data from its buffer as fast as it's arriving.



Result: Buffer overflows, packets are dropped. Wasteful and inefficient.

This is **Flow Control**: protecting the *receiver* from a fast sender.

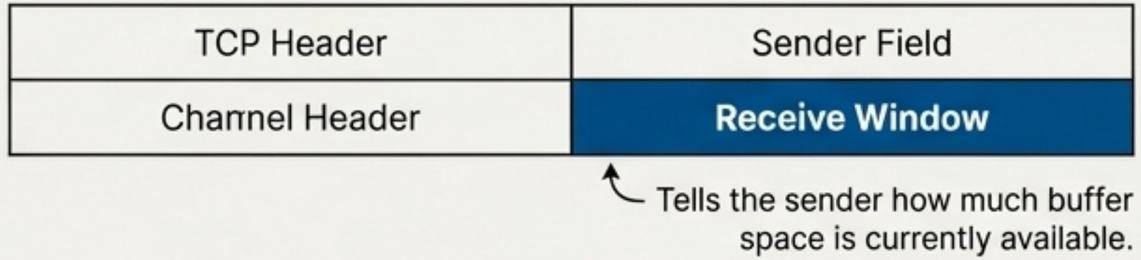
It is different from **Congestion Control**: protecting the *network* from too many senders.

NotebookLM

TCP's Solution: The Receive Window (rwnd)

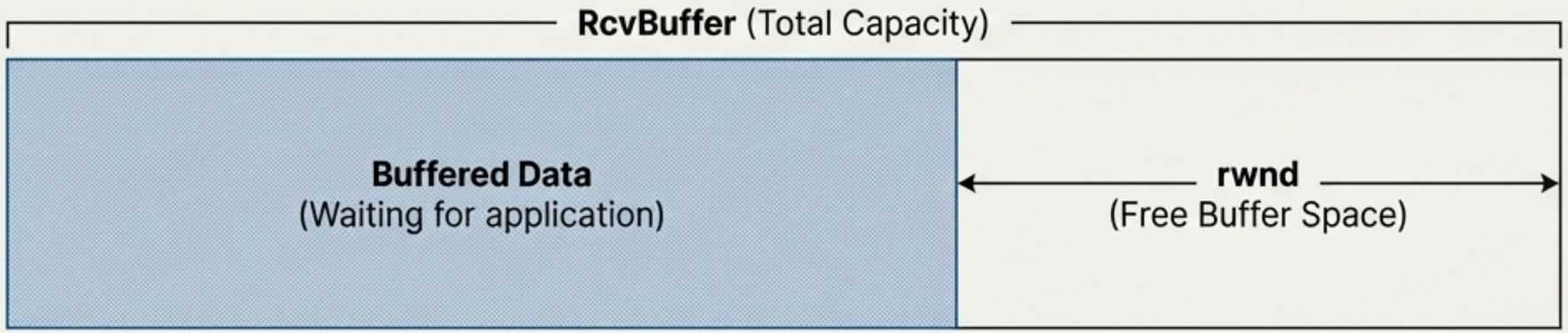
The Mechanism

The receiver tells the sender exactly how much free buffer space it has. This value is included in the receive window (rwnd) field of every TCP segment sent back to the sender.



The sender must ensure that the amount of unacknowledged ("in-flight") data is less than or equal to the last rwnd value received.

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$



This simple rule guarantees the receiver's buffer will never overflow.

NotebookLM

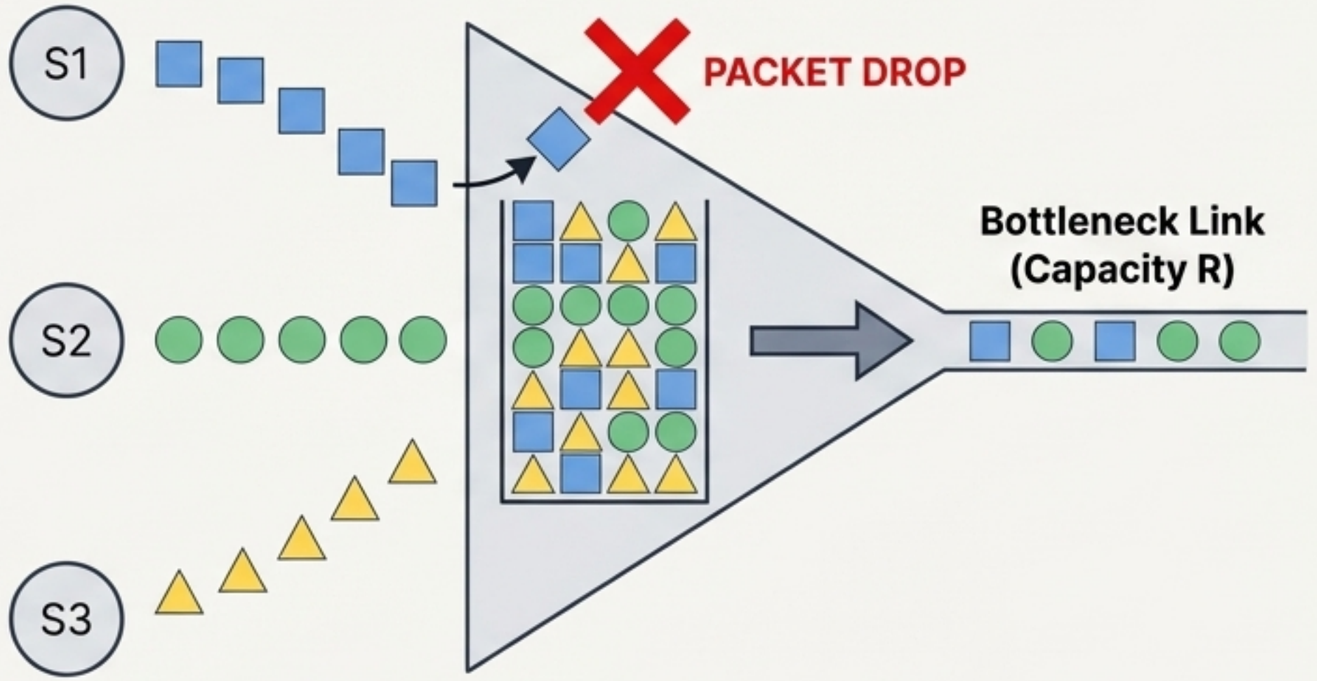
Challenge #4: The Network Traffic Jam

The Problem

- Multiple senders are all trying to send data through the same part of the network at the same time.
- The combined traffic is more than a router or link can handle.
- This overloaded router becomes the "**Bottleneck Link**".

The Consequences

1. **Long Delays**: Packets have to wait in the router's queue, increasing Round Trip Time (RTT).
2. **Packet Loss**: When the queue is full, the router has no choice but to drop incoming packets.



Key Insight
Once the bottleneck is congested, sending faster doesn't increase throughput. It only increases delay and packet loss.

NotebookLM

TCP’s Strategy: Probe Gently, Back Off Quickly

The Algorithm

AIMD (Additive Increase, Multiplicative Decrease)

Goal: A distributed algorithm for senders to share the network fairly without explicit coordination.

- **Additive Increase:** As long as packets get through, slowly increase the sending rate (by 1 segment per RTT). This is “gently probing for bandwidth”.
- **Multiplicative Decrease:** The moment packet loss is detected (a sign of congestion), slash the sending rate in half. This is “quickly backing off”.

NotebookLM

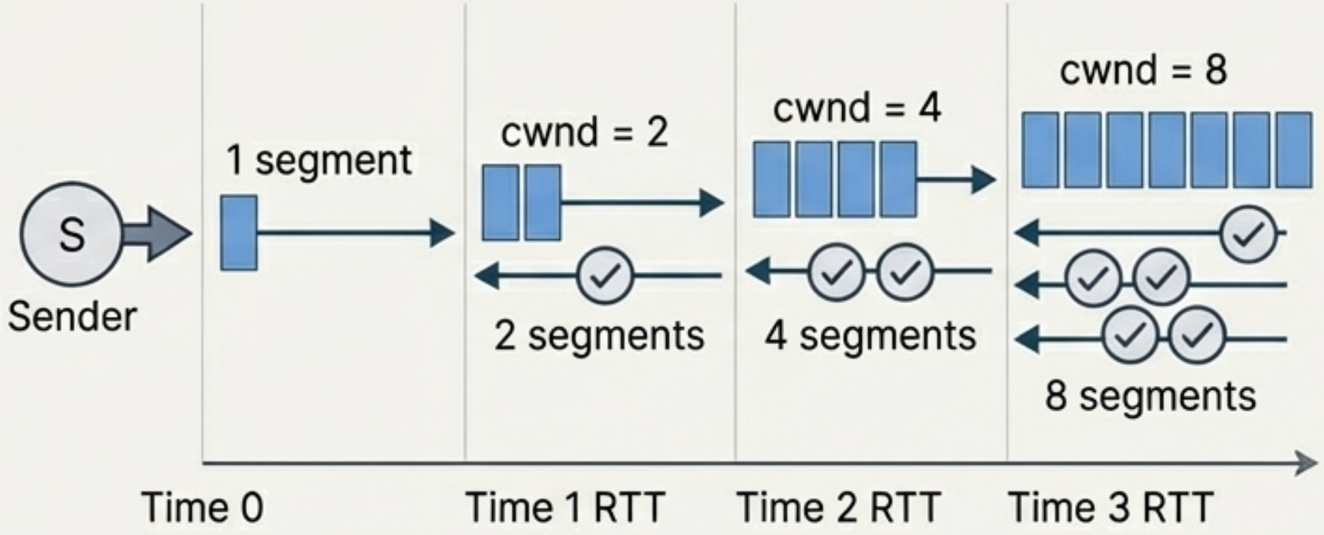
Starting the Race: TCP Slow Start

When a new connection begins, what is a safe initial sending rate? The sender has no idea what the network capacity is.

Start slow, but ramp up aggressively.

The Slow Start Algorithm

- Initialize the congestion window (*cwnd*) to **1 MSS** (Maximum Segment Size).
- For every ACK received, double the `*cwnd*`.
- This results in the sending rate doubling every Round Trip Time (RTT).
- **When does it stop?** The exponential growth continues until the first loss event, or until the window reaches a threshold (*ssthresh*), at which point it switches to the linear growth of AIMD.



The name is misleading; Slow Start ramps up exponentially fast to find the available bandwidth.

NotebookLM

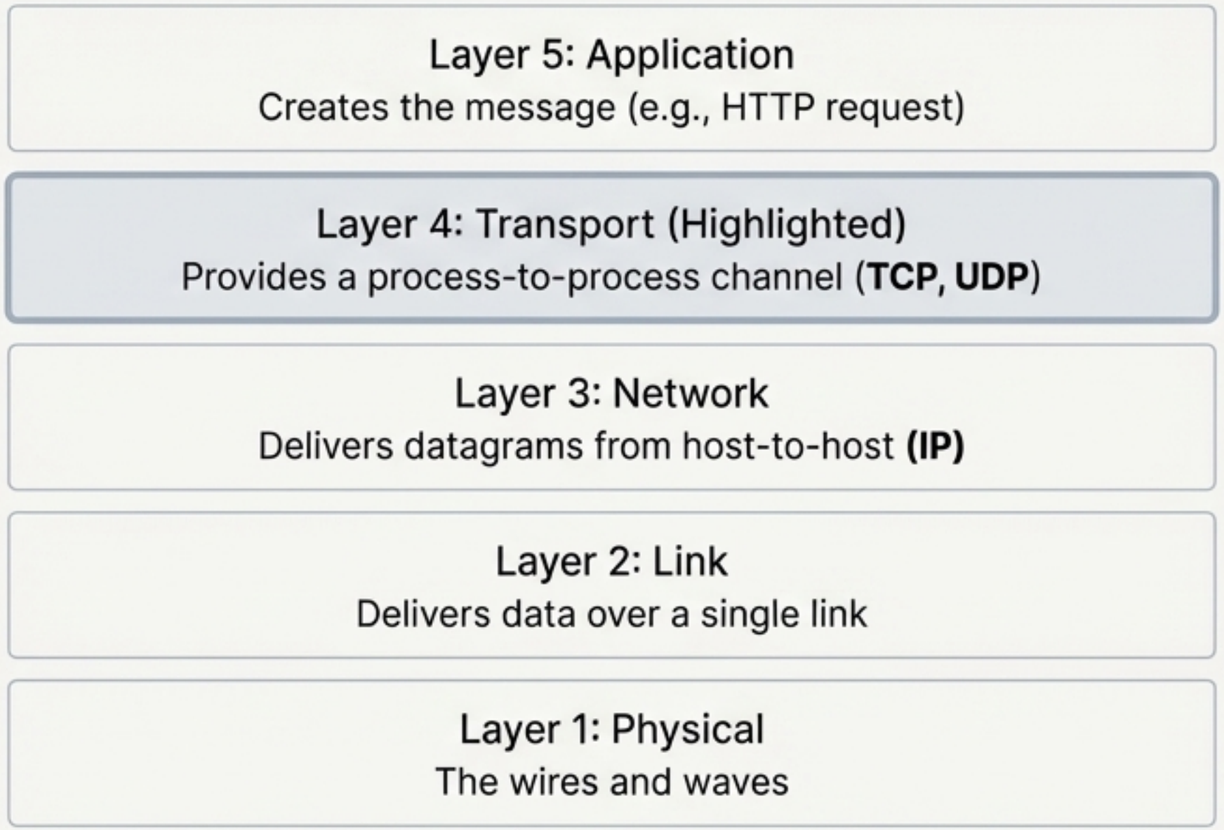
The Tale of Two Protocols: Choosing Your Champion

UDP (The Sprinter)	TCP (The Marathon Runner)
Connection: Connectionless (no handshake)	**Connection:** Connection-Oriented (3-way handshake)
Reliability: Unreliable (“Best Effort”)	**Reliability:** High (ACKs, retransmissions)
Ordering: Unordered	**Ordering:** In-order delivery guaranteed
Error Check: Basic (Checksum)	**Error Check:** Robust (Checksum)
Flow Control: No	**Flow Control:** Yes (Receive Window)
Congestion Control: No	**Congestion Control:** Yes (AIMD, Slow Start)
Header Size: Small (8 bytes)	**Header Size:** Larger (20+ bytes)
Best For: Streaming video/audio, online gaming, DNS, VoIP. *Speed is critical, some loss is ok.*	**Best For:** Web browsing (HTTP), file transfer (FTP), email (SMTP). *100% data integrity is essential.*

NotebookLM

The Transport Layer's Place in the World

The Internet is built in layers, each providing a service to the one above it.



The Network Layer (IP) offers a basic but powerful host-to-host delivery service. The Transport Layer enhances this service, transforming it into a communication channel for the actual applications running on those hosts. It's the crucial link that makes the internet's vast application ecosystem possible. It turns a house-to-house postal service into a personal conversation.

NotebookLM